FILE: src/app/techgen/layout.tsx

```
0001: import React from "react";
0002: import BaseLayout from "@/layout/BaseLayout";
0003: import { NavItem } from "@/lib/types/nav";
0004: import { GridIcon, DocsIcon, DownloadIcon, TableIcon } from "@/icons";
0005: import { SidebarProvider } from "@/context/SidebarContext";
0006: import TanStackProvider from "@/providers/TanStackProvider";
0007: import UserProvider from "@/providers/UserProvider";
0008:
0009: const navItems: NavItem[] = [
0010:   { icon: <GridIcon />, name: "Dashboard", path: "/techgen" },
0011:   {
0012:     icon: <TableIcon />,
0013:     name: "Applications Registry",
0014:     path: "/techgen/application-registry",
0015:   },
0016:   {
0017:     icon: <DocsIcon />,
0018:     name: "Application Guide",
0019:     path: "/techgen/application-guide",
0020:   },
0021:   {
0022:     icon: <DownloadIcon />,
0023:     name: "Application Documents",
0024:     path: "/techgen/application-document",
0025:   },
0026: ];
0027:
0028: export default function Layout({ children }: { children: React.ReactNode }) {
0029:   return (
0030:     <TanStackProvider>
0031:       <UserProvider>
0032:         <SidebarProvider>
0033:           <BaseLayout navItems={navItems}>{children}</BaseLayout>
0034:         </SidebarProvider>
0035:       </UserProvider>
0036:     </TanStackProvider>
0037:   );
0038: }
```

FILE: src/app/techgen/page.tsx

```
0001: "use client";
0002:
0003: import { MetricsTechgen } from "@/components/techgen/MetricsTechgen";
0004: import React, { useMemo } from "react";
0005: import StatusUpdatesPanel from "@/components/techgen/StatusUpdatesPanel";
0006: import { useGetUserApplicationIds } from "@/hooks/applications/useGetUserApplications";
0007: import Link from "next/link";
0008: import { PlusIcon } from "lucide-react";
0009:
0010: export default function TechgenDashboard() {
0011:   const { userApplicationIds } = useGetUserApplicationIds();
0012:
0013:   // children gets stable reference
0014:   const applicationIds = useMemo(() => {
0015:     return userApplicationIds
0016:       ?.map((app) => app.application_id)
0017:       .filter((id): id is string => id != null && id.trim() !== "");
0018:   }, [userApplicationIds]);
0019:
0020:   return (
0021:     <div className="space-y-6">
0022:       <div className="flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-between">
0023:         <div>
0024:           <h1 className="text-2xl font-semibold text-gray-800">Dashboard</h1>
0025:           <p className="mt-1 text-sm text-gray-500">
0026:             Track your applications and recent status updates.
```

```
0027:           </p>
0028:         </div>
0029:
0030:         <Link
0031:           href="/techgen/new-application"
0032:           className="bg-brand-500 hover:bg-brand-600 inline-flex h-11 items-center justify-center gap-2 rounded-
      | lg px-4 text-sm font-medium text-white"
0033:         >
0034:           <PlusIcon size={18} />
0035:           Add New Application
0036:         </Link>
0037:       </div>
0038:
0039:       <div className="grid grid-cols-12 gap-4 md:gap-8">
0040:         <div className="col-span-12 xl:col-span-8">
0041:           <MetricsTechgen />
0042:         </div>
0043:
0044:         <div className="col-span-12 xl:col-span-4">
0045:           <StatusUpdatesPanel applicationIds={applicationIds ?? []} />
0046:         </div>
0047:       </div>
0048:     </div>
0049:   );
0050: }
```

```
================================================================================================
FILE: src/app/techgen/application-registry/page.tsx
================================================================================================
0001: import React from "react";
0002: import ApplicationsTable from "@/components/common/ApplicationsTable";
0003:
0004: export default function TechgenApplicationRegistry() {
0005:   return (
0006:     <div>
0007:       <ApplicationsTable isTechgen={true} />
0008:     </div>
0009:   );
0010: }
```

```
================================================================================================
FILE: src/app/techgen/application-guide/page.tsx
================================================================================================
0001: import ApplicationGuidePage from "@/components/common/ApplicationGuidePage";
0002:
0003: export default function GuestApplicationGuidePage() {
0004:   return <ApplicationGuidePage isApplicant={true} />;
0005: }
```

```
================================================================================================
FILE: src/app/techgen/application-document/page.tsx
================================================================================================
0001: import React from "react";
0002: import ApplicationDocuments from "@/components/common/ApplicationDocuments";
0003:
0004: export default function TechGenApplicationDocs() {
0005:   return <ApplicationDocuments />;
0006: }
```

```
================================================================================================
FILE: src/app/techgen/notifications/page.tsx
================================================================================================
0001: import ViewAllNotifications from "@/components/common/ViewAllNotifications";
0002:
0003: export default function NotificationsPage() {
0004:   return <ViewAllNotifications />;
0005: }
```

```
================================================================================================
FILE: src/app/techgen/new-application/page.tsx
================================================================================================
0001: "use client";
```

```
0002:
0003: import NewApplicationFlow from "@/components/application/NewApplicationFlow";
0004: import { TECHGEN_NEW_APPLICATION_COPY } from "@/lib/constants/new-application";
0005:
0006: export default function TechGenNewApplicationPage() {
0007:   return (
0008:     <NewApplicationFlow
0009:       startApplicationPath="/techgen/start-application"
0010:       copy={TECHGEN_NEW_APPLICATION_COPY}
0011:     />
0012:   );
0013: }
```

```
================================================================================================
FILE: src/app/techgen/start-application/page.tsx
================================================================================================
0001: import StartApplicationPageClient from "./StartApplicationPageClient";
0002: import { IpType, IP_TYPES } from "@/lib/types/ip";
0003:
0004: export default async function Page({
0005:   searchParams,
0006: }: {
0007:   searchParams: Promise<{ ipType?: string }>;
0008: }) {
0009:   const { ipType } = await searchParams;
0010:
0011:   if (!ipType || !IP_TYPES.includes(ipType as IpType)) {
0012:     return (
0013:       <div className="flex w-full flex-1 flex-row items-center justify-center gap-2">
0014:         Invalid IP Type. Please go back and select a valid IP Type.
0015:       </div>
0016:     );
0017:   }
0018:
0019:   return <StartApplicationPageClient ipType={ipType as IpType} />;
0020: }
```

```
================================================================================================
FILE: src/app/techgen/start-application/StartApplicationPageClient.tsx
================================================================================================
0001: "use client";
0002:
0003: import { useEffect, useState } from "react";
0004: import { useRouter } from "next/navigation";
0005: import { AttachmentType, InventorType } from "@/lib/types/application";
0006: import { IpType } from "@/lib/types/ip";
0007: import { FUNDING_SOURCE_OPTIONS } from "@/lib/constants/funding-sources";
0008: import useAddVerifiedInventorsModal from "@/hooks/useAddVerifiedInventorModal";
0009: import { useCreateApplication } from "@/hooks/applications/useCreateApplication";
0010: import { useUploadFile } from "@/hooks/attachments/useUploadFile";
0011: import { useAtomValue } from "jotai";
0012: import { userAtom } from "@/atom-states/user";
0013: import { useConfirm } from "@/hooks/useConfirm";
0014:
0015: import { Input } from "@/components/ui/input";
0016: import { ScrollArea } from "@/components/ui/scroll-area";
0017: import {
0018:   Select,
0019:   SelectContent,
0020:   SelectItem,
0021:   SelectTrigger,
0022:   SelectValue,
0023: } from "@/components/ui/select";
0024: import FileUploader from "@/components/common/FileUploader";
0025: import { ArrowLeft, X, VerifiedIcon, Loader } from "lucide-react";
0026: import { Button } from "@/components/ui/button";
0027: import Hint from "@/components/common/Tooltip";
0028: import { toast } from "sonner";
0029:
0030: type ExtendedAttachmentType = AttachmentType["Insert"] & {
0031:   fileObject?: File;
0032: };
```

```
0033:
0034: interface StartApplicationPageClientProps {
0035:   ipType: IpType;
0036: }
0037:
0038: export default function StartApplicationPageClient({
0039:   ipType,
0040: }: StartApplicationPageClientProps) {
0041:   const router = useRouter();
0042:   const confirm = useConfirm();
0043:   const { isLoading: isCreatingApp, createApp } = useCreateApplication();
0044:   const { isLoading: isUploadingFiles, uploadFile } = useUploadFile();
0045:   const {
0046:     isOpen: isAddVerifiedInventorModalOpen,
0047:     inventor,
0048:     openModal: openAddVerifiedInventorModal,
0049:     setExcludedUIDs,
0050:   } = useAddVerifiedInventorsModal();
0051:   const user = useAtomValue(userAtom);
0052:   const [fileItems, setFileItems] = useState<ExtendedAttachmentType[]>([]);
0053:   const [inventors, setInventors] = useState<InventorType["Insert"][]>([]);
0054:   const [projectTitle, setProjectTitle] = useState("");
0055:   const [fundingSource, setFundingSource] = useState("");
0056:   const [otherFundingSource, setOtherFundingSource] = useState("");
0057:   const isOtherFundingSource = fundingSource === "Others";
0058:   const resolvedFundingSource = isOtherFundingSource
0059:     ? otherFundingSource.trim()
0060:     : fundingSource.trim();
0061:
0062:   function handleBack() {
0063:     router.back();
0064:   }
0065:
0066:   function removeInventor(index: number, techgenId: string | undefined | null) {
0067:     if (techgenId === user?.id) return;
0068:     setInventors((prev) => prev.filter((_, i) => i !== index));
0069:     if (techgenId) {
0070:       setExcludedUIDs((prev) => prev.filter((id) => id !== techgenId));
0071:     }
0072:   }
0073:
0074:   function addInventor() {
0075:     openAddVerifiedInventorModal();
0076:   }
0077:
0078:   async function handleSubmit() {
0079:     const isConfirmed = await confirm({
0080:       title: "Confirm Submission",
0081:       message:
0082:         "Are you sure you want to submit this application? Please ensure that all details are correct before
      | confirming.",
0083:     });
0084:
0085:     if (!isConfirmed) return;
0086:     if (projectTitle.trim() === "" || resolvedFundingSource === "") return;
0087:
0088:     toast.promise(createAndUpload(), {
0089:       error: (e: Error) => {
0090:         if (e.message.includes("inventors_application_id_email_key")) {
0091:           return "Error: Duplicate email address. Please remove the duplicate collaborator and try again.";
0092:         }
0093:         return "Failed to create application. Please check the instructions and try again.";
0094:       },
0095:     });
0096:   }
0097:
0098:   async function createAndUpload() {
0099:     if (user === null || user === undefined) {
0100:       throw new Error("User information is not available.");
0101:     }
0102:
0103:     const collaboratorInventors = inventors.filter(
```

```
      (inventorItem) => inventorItem.techgen_id !== user.id,
    );

    const appId = await createApp({
      applicationData: {
        project_title: projectTitle,
        ip_type: ipType,
        funding_source: resolvedFundingSource,
      },
      inventorsData: collaboratorInventors,
    });

    await handleUpload(appId, fileItems);
    router.push(`/techgen/view-application?applicationID=${appId}`);
  }

  async function handleUpload(
    appId: string,
    fileItems: ExtendedAttachmentType[],
  ) {
    for (const item of fileItems) {
      await uploadFile(
        { file: item, appId },
        {
          onSettled: handleSettled,
        },
      );
    }
  }

  function handleSettled() {
    setFileItems((prev) => {
      const remainingItems = prev.filter((_, index) => index !== 0);
      return remainingItems;
    });
  }

  useEffect(() => {
    if (inventor === null) return;

    setInventors((prev) => [...prev, inventor]);

    if (inventor.techgen_id !== undefined && inventor.techgen_id !== null) {
      setExcludedUIDs((prev) => [...prev, inventor.techgen_id as string]);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [inventor, isAddVerifiedInventorModalOpen]);

  useEffect(() => {
    if (user === null || user === undefined) return;
    setInventors((prev) => {
      const hasCurrentUser = prev.some(
        (inventorItem) => inventorItem.techgen_id === user.id,
      );

      if (hasCurrentUser) return prev;

      return [
        {
          application_id: "",
          techgen_id: user.id,
          full_name: user.full_name,
          email: user.email,
          college_code: user.college_code,
          other_college_name: user.other_college_name,
          external_institution: user.external_institution,
          status: "member",
        },
        ...prev,
      ];
    });
    setExcludedUIDs((prev) =>
```

```
      prev.includes(user.id) ? prev : [...prev, user.id],
    );
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [user]);

  if (user === null || user === undefined) {
    return (
      <div className="flex w-full flex-1 flex-row items-center justify-center gap-2">
        <span className="text-lg font-medium">Loading user information...</span>
        <Loader className="animate-spin" />
      </div>
    );
  }

  if (isCreatingApp) {
    return (
      <div className="flex w-full flex-1 flex-row items-center justify-center gap-2">
        <span className="text-lg font-medium">Creating application...</span>
        <Loader className="animate-spin" />
      </div>
    );
  }

  if (isUploadingFiles) {
    return (
      <div className="flex w-full flex-1 flex-row items-center justify-center gap-2">
        <span className="text-lg font-medium">Uploading files...</span>
        <Loader className="animate-spin" />
      </div>
    );
  }

  return (
    <div className="flex justify-center">
      <div className="relative mx-auto max-w-6xl space-y-6 px-4 py-8">
        <button
          type="button"
          onClick={handleBack}
          aria-label="Return to previous page"
          className="absolute top-8 left-0 inline-flex h-9 w-9 items-center justify-center rounded-full border
    | border-slate-200 bg-white hover:bg-slate-50 focus:outline-none"
        >
          <ArrowLeft size={18} className="text-slate-700" />
        </button>

        <header>
          <h1 className="mb-4 px-12 text-3xl font-semibold">
            Fill-up application details
          </h1>
        </header>

        <div>
          <h2 className="text-2xl font-medium">A. Information Details</h2>
          <p className="mt-1 text-lg text-slate-500">
            Provide the title of your research or project. If there is a funding
            source for this IP, please indicate it here.
          </p>
        </div>

        <span className="text-lg font-medium">
          Research/Project title <span className="text-red-500">*</span>
        </span>
        <Input
          placeholder="e.g., A study on the effectiveness of IRIS in managing intellectual property"
          className="mt-1 h-12 text-lg!"
          value={projectTitle}
          onChange={(e) => {
            setProjectTitle(e.target.value);
          }}
        />

        <label className="flex w-full flex-col gap-1">
```

```
          <span className="text-lg font-medium">
            Funding source <span className="text-red-500">*</span>
          </span>
          <Select
            value={fundingSource}
            onValueChange={(value) => {
              setFundingSource(value);
              if (value !== "Others") {
                setOtherFundingSource("");
              }
            }}
          >
            <SelectTrigger className="mt-1 h-12 w-full text-left text-base sm:text-lg">
              <SelectValue placeholder="Select funding source" />
            </SelectTrigger>

            <SelectContent
              position="popper"
              side="bottom"
              className="z-9999 max-h-60 overflow-y-auto"
            >
              {FUNDING_SOURCE_OPTIONS.map((option) => (
                <SelectItem
                  key={option}
                  value={option}
                  className="cursor-pointer"
                >
                  {option}
                </SelectItem>
              ))}
            </SelectContent>
          </Select>
        </label>

        {isOtherFundingSource && (
          <label className="flex w-full flex-col gap-1">
            <span className="text-lg font-medium">
              Other funding source <span className="text-red-500">*</span>
            </span>
            <Input
              placeholder="Type the funding source"
              className="mt-1 h-12 text-lg!"
              value={otherFundingSource}
              onChange={(e) => {
                setOtherFundingSource(e.target.value);
              }}
            />
          </label>
        )}

        <div>
          <h2 className="text-2xl font-medium">B. Collaborators</h2>
          <p className="mt-1 text-lg text-slate-500">
            List all the collaborators for this application.{" "}
            <b>You are automatically listed</b> as a technology generator so
            exclude yourself from this list.
          </p>
        </div>

        <ScrollArea className="h-[300px] rounded-md border p-2 pr-4">
          {inventors.length === 0 && (
            <div className="text-muted-foreground mt-28 flex h-full w-full items-center justify-center text-
    | center text-lg">
              No technology generator collaborators added yet.
            </div>
          )}

          {inventors.map((inventor, index) => (
            <div
              key={index + inventor.full_name}
              className="bg-card text-card-foreground mb-2 flex flex-col gap-2 rounded-lg border p-3 shadow-sm"
            >
```

```
              <div className="flex items-start justify-between gap-3">
                <div className="flex flex-col items-start gap-1 overflow-hidden">
                  <span className="block truncate text-lg font-medium">
                    {inventor.full_name}
                  </span>

                  <div className="text-muted-foreground text-md">
                    {inventor.email}
                  </div>

                  <div className="flex flex-row items-center gap-2 align-middle text-sky-700">
                    <Hint
                      label={
                        inventor.external_institution ??
                        inventor.college_code ??
                        inventor.other_college_name ??
                        ""
                      }
                    >
                      <span className="block max-w-32 truncate rounded-full bg-slate-100 px-2 py-0.5 text-sm
    | font-medium text-slate-700 uppercase">
                        {inventor.external_institution ??
                          inventor.college_code ??
                          inventor.other_college_name}
                      </span>
                    </Hint>

                    {inventor.techgen_id !== undefined &&
                      inventor.techgen_id !== null && (
                        <div className="flex flex-row items-center gap-2 px-2 align-middle text-sm font-medium
    | text-sky-700">
                          {inventor.techgen_id === user.id ? "You" : "Verified"}
                          <VerifiedIcon size={20} />
                        </div>
                      )}
                  </div>
                </div>

                <Button
                  variant="ghost"
                  size="icon"
                  className="text-muted-foreground hover:text-destructive h-8 w-8"
                  onClick={() => removeInventor(index, inventor.techgen_id)}
                  disabled={inventor.techgen_id === user.id}
                >
                  <X className="h-4 w-4" />
                </Button>
              </div>
            </div>
          ))}
        </ScrollArea>

        <button
          type="button"
          onClick={addInventor}
          className="h-10 w-full items-center rounded-md bg-sky-600 px-4 py-2 text-center text-sm font-semibold
    | text-white shadow-sm disabled:cursor-not-allowed disabled:bg-slate-300"
        >
          List technology generator collaborators
        </button>

        <div>
          <h2 className="text-2xl font-medium">C. Relevant attachments</h2>
          <p className="mt-1 text-lg text-slate-500">
            <span className="block">
              Please attach only <b>one (1) PDF file</b> containing the
              following information:
            </span>
            <span className="mt-2 block">
              (1) The appropriate disclosure form
              <br />
              (2) Any relevant supporting documents (e.g., research paper,
```

```
0387:               prototype design, copyright material, images, figures, etc.)
0388:             </span>
0389:             <span className="mt-2 block">
0390:               Compile or merge into <b>one (1) PDF file</b> and upload here.
0391:             </span>
0392:           </p>
0393:         </div>
0394:
0395:         <div className="flex w-full justify-center p-4">
0396:           <FileUploader
0397:             items={fileItems}
0398:             setItems={setFileItems}
0399:             maxFileCount={1}
0400:             acceptedFileTypes={{
0401:               "application/pdf": [".pdf"],
0402:             }}
0403:           />
0404:         </div>
0405:
0406:         <button
0407:           type="button"
0408:           onClick={handleSubmit}
0409:           disabled={
0410:             projectTitle.trim() === "" ||
0411:             resolvedFundingSource === "" ||
0412:             fileItems.length === 0
0413:           }
0414:           className="h-10 w-full items-center rounded-md bg-sky-600 px-4 py-2 text-center text-sm font-semibold
      | text-white shadow-sm disabled:cursor-not-allowed disabled:bg-slate-300"
0415:         >
0416:           Submit Application
0417:         </button>
0418:       </div>
0419:     </div>
0420:   );
0421: }
```

```
================================================================================================
FILE: src/app/techgen/view-application/page.tsx
================================================================================================
0001: import TechgenViewApplicationPageClient from "./TechgenViewApplicationPageClient";
0002:
0003: interface TechgenViewApplicationPageProps {
0004:   searchParams: Promise<{
0005:     applicationID?: string;
0006:   }>;
0007: }
0008:
0009: export default async function TechgenViewApplicationPage({
0010:   searchParams,
0011: }: TechgenViewApplicationPageProps) {
0012:   const params = await searchParams;
0013:   const applicationId = params.applicationID ?? "";
0014:
0015:   return <TechgenViewApplicationPageClient applicationId={applicationId} />;
0016: }
```

```
================================================================================================
FILE: src/app/techgen/view-application/TechgenViewApplicationPageClient.tsx
================================================================================================
0001: "use client";
0002:
0003: import ApplicationView from "@/components/application/ApplicationView";
0004: import { useGetAppById } from "@/hooks/applications/useGetApplicationById";
0005: import { Loader } from "lucide-react";
0006:
0007: interface TechgenViewApplicationPageClientProps {
0008:   applicationId: string;
0009: }
0010:
0011: function TechgenViewApplicationPageClient({
0012:   applicationId,
```

```
0013: }: TechgenViewApplicationPageClientProps) {
0014:   const { application, isLoading, isFetched } = useGetAppById({
0015:     appId: applicationId,
0016:   });
0017:
0018:   if ((isLoading || !isFetched) && !application) {
0019:     return (
0020:       <div className="flex w-full flex-1 flex-row items-center justify-center gap-2">
0021:         Fetching Application...
0022:         <Loader className="animate-spin" />
0023:       </div>
0024:     );
0025:   }
0026:
0027:   if (!application) {
0028:     return (
0029:       <div className="flex w-full flex-1 flex-row items-center justify-center gap-2">
0030:         Application not found.
0031:       </div>
0032:     );
0033:   }
0034:
0035:   return <ApplicationView mode="applicant" initialApplication={application} />;
0036: }
0037:
0038: export default TechgenViewApplicationPageClient;
```

```
================================================================================================
FILE: src/components/techgen/MetricsTechgen.tsx
================================================================================================
0001: "use client";
0002:
0003: import React, { useMemo } from "react";
0004: import {
0005:   FileText,
0006:   Send,
0007:   Clock3,
0008:   CheckCircle2,
0009:   Undo2,
0010:   ArrowDown,
0011:   Shield,
0012:   Shapes,
0013:   Palette,
0014:   BadgeCheck,
0015:   Copyright,
0016:   Inbox,
0017:   CalendarRange,
0018:   Layers3,
0019: } from "lucide-react";
0020: import { useGetDashboardAnalyticsTechgen } from "@/hooks/views/useGetDashboardAnalyticsTechgen";
0021: import { STATUS_ORDER } from "@/lib/dashboard/dashboard-summary";
0022: import { IpType, IP_TYPES } from "@/lib/types/ip";
0023: import { ipTypeToTitle } from "@/lib/helper/get-ip-title";
0024:
0025: type MetricCardProps = {
0026:   title: string;
0027:   value: number;
0028:   subtitle?: string;
0029:   icon: React.ElementType;
0030:   emptyText?: string;
0031:   isEmpty?: boolean;
0032: };
0033:
0034: function MetricCard({
0035:   title,
0036:   value,
0037:   subtitle,
0038:   icon: Icon,
0039:   emptyText = "No data yet",
0040:   isEmpty = false,
0041: }: MetricCardProps) {
0042:   return (
```

```
0591:             variant="outline"
0592:             size="sm"
0593:             onClick={() => handlePageChange(currentPage - 1)}
0594:             disabled={currentPage === 1}
0595:           >
0596:             Previous
0597:           </Button>
0598:           <div className="flex gap-2">
0599:             {generatePagination(currentPage, totalPages).map((page, i) =>
0600:               page === "..." ? (
0601:                 <span
0602:                   key={`ellipsis-${i}-${page}`}
0603:                   className="flex h-8 w-8 items-center justify-center text-sm text-gray-500"
0604:                 >
0605:                   ...
0606:                 </span>
0607:               ) : (
0608:                 <Button
0609:                   key={page.toString() + "-page-button"}
0610:                   variant={currentPage === page ? "primary" : "outline"}
0611:                   size="sm"
0612:                   onClick={() => handlePageChange(page as number)}
0613:                 >
0614:                   {page}
0615:                 </Button>
0616:               ),
0617:             )}
0618:           </div>
0619:           <Button
0620:             variant="outline"
0621:             size="sm"
0622:             onClick={() => handlePageChange(currentPage + 1)}
0623:             disabled={currentPage === totalPages}
0624:           >
0625:             Next
0626:           </Button>
0627:         </div>
0628:       )}
0629:     </div>
0630:   );
0631: }
```

```
================================================================================================
FILE: src/components/common/ApplicationGuidePage.tsx
================================================================================================
0001: "use client";
0002:
0003: import ApplicationFlowChart from "@/components/common/ApplicationFlowChart";
0004: import CallToAction from "@/components/common/CallToAction";
0005: import ApplicationGuide from "./ApplicationGuide";
0006:
0007: interface ApplicationGuidePageProps {
0008:   isApplicant?: boolean;
0009: }
0010:
0011: export default function ApplicationGuidePage(props: ApplicationGuidePageProps) {
0012:   const { isApplicant = false } = props;
0013:
0014:   return (
0015:     <div className="space-y-6">
0016:       <section className="max-w-3xl">
0017:         <p className="text-sm font-medium text-blue-700">Getting started</p>
0018:         <h1 className="mt-1 text-2xl font-semibold text-slate-900 sm:text-3xl">
0019:           Application Guide
0020:         </h1>
0021:         <p className="mt-2 text-sm text-slate-600 sm:text-base">
0022:           Learn the basic steps, required documents, and process flow before
0023:           applying for IP protection through IRIS.
0024:         </p>
0025:       </section>
0026:
0027:       <div className="grid grid-cols-1 gap-6 xl:grid-cols-[minmax(0,1fr)_320px]">
```

```
0028:         <ApplicationGuide />
0029:
0030:         <aside className="space-y-4">
0031:           <CallToAction
0032:             title={isApplicant ? "Ready to begin?" : "Ready to apply?"}
0033:             description={
0034:               isApplicant
0035:                 ? "Start a new IP application once you?re ready."
0036:                 : "Sign in to IRIS when you?re ready to begin your application."
0037:             }
0038:             primaryLabel={
0039:               isApplicant
0040:                 ? "Start a New IP Application"
0041:                 : "Apply for IP Protection"
0042:             }
0043:             primaryHref={isApplicant ? "/techgen/new-application" : "/signin"}
0044:             showSecondaryButton={false}
0045:           />
0046:         </aside>
0047:       </div>
0048:
0049:       <ApplicationFlowChart />
0050:     </div>
0051:   );
0052: }
```

```
================================================================================================
FILE: src/components/common/ApplicationDocuments.tsx
================================================================================================
0001: "use client";
0002:
0003: import { useState } from "react";
0004: import clsx from "clsx";
0005: import { CircleHelp, ShieldCheck } from "lucide-react";
0006:
0007: import CallToAction from "@/components/common/CallToAction";
0008: import DisclosureFormActions from "@/components/application/DisclosureFormActions";
0009: import { ipTypeToTitle } from "@/lib/helper/get-ip-title";
0010: import { IpType, IP_TYPES } from "@/lib/types/ip";
0011: import { cn } from "@/lib/utils";
0012: import Button from "../ui/button/Button";
0013:
0014: interface PropsInterface {
0015:   isApplicant?: boolean;
0016:   isAdmin?: boolean;
0017: }
0018:
0019: export default function ApplicationDocuments(props: PropsInterface) {
0020:   const { isApplicant = true, isAdmin = false } = props;
0021:   const [activeIpType, setActiveIpType] = useState<IpType>("patent");
0022:
0023:   return (
0024:     <div className="space-y-6">
0025:       <section className="max-w-3xl">
0026:         <p className="text-sm font-medium text-blue-700">Document hub</p>
0027:         <h1 className="mt-1 text-2xl font-semibold text-slate-900 sm:text-3xl">
0028:           Application Documents
0029:         </h1>
0030:         <p className="mt-2 text-sm text-slate-600 sm:text-base">
0031:           Browse forms and reference files by IP type before starting your
0032:           application in IRIS.
0033:         </p>
0034:       </section>
0035:
0036:       <div className="grid grid-cols-1 gap-6 xl:grid-cols-[minmax(0,1fr)_320px]">
0037:         <section className="rounded-3xl border border-slate-200 bg-white p-4 sm:p-6">
0038:           <div className="rounded-2xl bg-slate-100 p-1.5">
0039:             <div className="flex gap-2 overflow-x-auto [-ms-overflow-style:none] [scrollbar-width:none]
      | [&::-webkit-scrollbar]:hidden">
0040:               {IP_TYPES.map((ipType) => {
0041:                 const isActive = ipType === activeIpType;
0042:
```

```
                return (
                  <button
                    key={ipType}
                    type="button"
                    onClick={() => setActiveIpType(ipType)}
                    className={clsx(
                      "shrink-0 rounded-xl px-4 py-2.5 text-sm font-medium whitespace-nowrap transition",
                      isActive
                        ? "bg-white text-blue-700 ring-1 ring-blue-100 ring-inset"
                        : "text-slate-600 hover:bg-white/70 hover:text-slate-900",
                    )}
                  >
                    {ipTypeToTitle(ipType)}
                  </button>
                );
              })}
            </div>
          </div>

          <div className="mt-5">
            <DisclosureFormActions
              title={ipTypeToTitle(activeIpType)}
              description=""
              finalIpType={activeIpType}
              canManageFiles={isAdmin}
              showHeader={false}
              showProceed={false}
              showFooterNote={false}
              filesTitle={null}
            />
          </div>
        </section>

        <aside
          className={`space-y-4 ${isApplicant ? "" : "flex flex-col-reverse justify-end gap-2"}`}
        >
          {isApplicant && (
            <CallToAction
              title="Ready to begin?"
              description="Start a new IP application once you?re ready."
              primaryLabel="Start a New IP Application"
              primaryHref="/techgen/new-application"
              showSecondaryButton={false}
            />
          )}

          <section className="rounded-3xl border border-slate-200 bg-white p-5">
            <div className="flex items-center gap-2">
              <div className="rounded-xl bg-slate-100 p-2">
                <ShieldCheck className="h-5 w-5 text-slate-700" />
              </div>
              <h2 className="text-base font-semibold text-slate-900">
                Reminders
              </h2>
            </div>

            <ul className="mt-3 space-y-2 text-sm text-slate-600">
              <li>Use files that match your selected IP type.</li>
              <li>Check the forms before filling them out.</li>
              <li>Prepare complete supporting requirements early.</li>
            </ul>
          </section>

          {!isAdmin && (
            <section
              className={cn(
                "bg-white p-5 transition-all duration-300",
                isApplicant
                  ? // Applying our new custom class from globals.css
                    "rounded-3xl border border-slate-200"
                  : "animate-sky-pulse rounded-3xl border-2",
              )}
```

```
            >
              <div className="flex items-center gap-2">
                <div className="rounded-xl bg-slate-100 p-2">
                  <CircleHelp className="h-5 w-5 text-slate-700" />
                </div>
                <h2 className="text-base font-semibold text-slate-900">
                  Need help?
                </h2>
              </div>

              <p className="mt-3 text-sm text-slate-600">
                {isApplicant
                  ? "You can review the files here first, then use the application wizard when you are ready to
    | begin."
                  : "You can review the files here to get familiar with the application process. You can also
    | check out the application wizard to find out which IP type is right for you."}
              </p>
              {!isApplicant && (
                <Button
                  size="md"
                  variant="primary"
                  onClick={() => {
                    globalThis.location.href = "/wizard";
                  }}
                  className="mt-4 w-full flex-1"
                >
                  Use Application Wizard
                </Button>
              )}
            </section>
          )}
        </aside>
      </div>
    </div>
  );
}
```

```
================================================================================================
FILE: src/components/common/ViewAllNotifications.tsx
================================================================================================
"use client";

import { useState } from "react";
import { useGetNotifications } from "@/hooks/notifications/useGetNotifications";
import { useMarkAsRead } from "@/hooks/notifications/useMarkAsRead";
import { formatDateTime, formatTime } from "@/lib/helper/format-date";
import { getNotificationApplicationLink } from "@/lib/helper/get-notification-application-link";
import { isToday } from "date-fns";
import Button from "@/components/ui/button/Button";
import { useRouter } from "next/navigation";

type FilterType = "all" | "unread" | "read";

interface ViewAllNotificationsProps {
  isAdmin?: boolean;
}

export default function ViewAllNotifications(props: ViewAllNotificationsProps) {
  const { isAdmin = false } = props;
  const { notifications, isLoading, isFetching } = useGetNotifications();
  const {
    markNotificationAsRead,
    markAllNotifcationsAsRead,
    isMarkingOne,
    isMarkingAll,
  } = useMarkAsRead();

  const [filter, setFilter] = useState<FilterType>("all");
  const router = useRouter();

  const unreadCount =
    notifications?.filter((n) => n.read_at === null).length ?? 0;
```

```
0033:
0034:   async function handleNotificationClick(
0035:     notifId: string,
0036:     readAt: null | string,
0037:     applicationId: string | null,
0038:   ) {
0039:     if (!readAt) {
0040:       await markNotificationAsRead({ notifId });
0041:     }
0042:
0043:     const href = getNotificationApplicationLink(applicationId, isAdmin);
0044:     if (href) router.push(href);
0045:   }
0046:
0047:   async function markAllAsRead() {
0048:     const unread = notifications?.filter((n) => n.read_at === null) ?? [];
0049:     if (unread.length === 0) return;
0050:
0051:     await markAllNotifcationsAsRead(unread);
0052:   }
0053:
0054:   const filtered = notifications?.filter((n) => {
0055:     if (filter === "unread") return n.read_at === null;
0056:     if (filter === "read") return n.read_at !== null;
0057:     return true;
0058:   });
0059:
0060:   return (
0061:     <div className="min-h-screen w-full bg-white px-6 py-8 transition-colors sm:px-8 lg:px-12">
0062:       <div className="mx-auto max-w-3xl">
0063:         <div className="xsm:flex-row relative mb-6 flex flex-col items-start justify-between gap-2">
0064:           <div className="xsm:gap-6 flex flex-row gap-4 sm:gap-10">
0065:             <button
0066:               onClick={() => router.back()}
0067:               className="mb-4 inline-flex h-10 w-10 items-center justify-center rounded-full border border-
      | gray-200 bg-white text-gray-500 transition-colors hover:bg-gray-50 lg:absolute lg:-left-20 lg:mb-0"
0068:               aria-label="Back"
0069:               title="Back"
0070:             >
0071:               <svg
0072:                 width="18"
0073:                 height="18"
0074:                 viewBox="0 0 24 24"
0075:                 fill="none"
0076:                 xmlns="http://www.w3.org/2000/svg"
0077:               >
0078:                 <path
0079:                   d="M19 12H5M5 12L12 19M5 12L12 5"
0080:                   stroke="currentColor"
0081:                   strokeWidth="2"
0082:                   strokeLinecap="round"
0083:                   strokeLinejoin="round"
0084:                 />
0085:               </svg>
0086:             </button>
0087:
0088:             <div>
0089:               <h1 className="text-2xl font-semibold text-gray-900">
0090:                 Notifications
0091:               </h1>
0092:
0093:               <div className="mt-1 flex items-center gap-2">
0094:                 {unreadCount > 0 ? (
0095:                   <p className="text-sm text-gray-500">{unreadCount} unread</p>
0096:                 ) : (
0097:                   <p className="text-sm text-gray-500">All caught up</p>
0098:                 )}
0099:
0100:                 {!isLoading && isFetching && (
0101:                   <>
0102:                     <span className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-gray-300
      |  border-t-gray-500" />
```

```
                    <span className="text-xs text-gray-400">Refreshing...</span>
                  </>
                )}
              </div>
            </div>
          </div>

          {unreadCount > 0 ? (
            <Button
              size="sm"
              variant="primary"
              onClick={markAllAsRead}
              disabled={isMarkingAll || notifications?.length === 0}
            >
              {isMarkingAll ? "Marking all as read..." : "Mark all as read"}
            </Button>
          ) : (
            <div className="w-px" />
          )}
        </div>

        <div className="mb-4">
          <div className="inline-flex w-full rounded-xl border border-gray-200 bg-gray-50 p-1">
            {(["all", "unread", "read"] as FilterType[]).map((tab) => {
              const active = filter === tab;

              return (
                <button
                  key={tab}
                  onClick={() => setFilter(tab)}
                  disabled={isLoading}
                  className={[
                    "relative flex-1 overflow-visible rounded-lg px-3 py-2 text-sm font-semibold capitalize | transition",
                    "focus:ring-2 focus:ring-gray-200 focus:ring-offset-1 focus:outline-none",
                    active
                      ? "bg-white text-gray-900 shadow-sm"
                      : "text-gray-600 hover:text-gray-900",
                    isLoading ? "cursor-not-allowed opacity-60" : "",
                  ].join(" ")}
                >
                  {tab}

                  {tab === "unread" && unreadCount > 0 && (
                    <span className="absolute -top-1 right-1 flex h-5 min-w-[20px] animate-pulse items-center | justify-center rounded-full bg-orange-500 px-1 text-xs leading-none font-semibold text-white shadow-sm">
                      {unreadCount}
                    </span>
                  )}
                </button>
              );
            })}
          </div>
        </div>

        <ul className="flex flex-col gap-2">
          {isLoading ? (
            [...Array(5)].map((_, i) => (
              <li
                key={i}
                className="flex animate-pulse gap-3 rounded-lg bg-gray-50 p-4 transition-colors"
              >
                <div className="h-10 w-10 rounded-full bg-gray-200" />
                <div className="flex-1 space-y-2 pt-1">
                  <div className="h-3 w-2/3 rounded bg-gray-200" />
                  <div className="h-3 w-full rounded bg-gray-200" />
                  <div className="h-3 w-1/3 rounded bg-gray-200" />
                </div>
              </li>
            ))
          ) : !filtered || filtered.length === 0 ? (
            <li className="flex flex-col items-center justify-center py-16 text-center">
```

```
              <p className="text-sm font-medium text-gray-700">
                {filter === "unread"
                  ? "No unread notifications"
                  : filter === "read"
                    ? "No read notifications"
                    : "No notifications yet"}
              </p>
              <p className="mt-1 text-sm text-gray-400">
                {filter === "all"
                  ? "You're all caught up!"
                  : 'Switch to "All" to see everything'}
              </p>
            </li>
          ) : (
            filtered.map((notif) => {
              const isUnread = notif.read_at === null;
              const isThisNotifLoading = isMarkingOne(notif.id);

              return (
                <li key={notif.id}>
                  <Button
                    onClick={() =>
                      handleNotificationClick(
                        notif.id,
                        notif.read_at,
                        notif.application_id,
                      )
                    }
                    size="md"
                    variant="outline"
                    disabled={isMarkingAll || isThisNotifLoading}
                    className={`group w-full justify-start gap-3 p-4 text-left transition-all duration-200 ${
                      isUnread
                        ? "bg-orange-50/40 hover:bg-orange-50/60 hover:shadow-sm hover:ring-1 hover:ring-
    | orange-200/70"
                        : "bg-white hover:bg-gray-50 hover:shadow-sm hover:ring-1 hover:ring-gray-200/70"
                    } ${
                      isMarkingAll || isThisNotifLoading
                        ? "cursor-not-allowed opacity-80"
                        : ""
                    }`}
                  >
                    <div className="flex w-full items-start gap-3">
                      <span className="mt-1.5 flex h-5 w-5 items-center justify-center">
                        {isThisNotifLoading ? (
                          <span className="h-3.5 w-3.5 animate-spin rounded-full border-2 border-gray-300
    | border-t-orange-500" />
                        ) : isUnread ? (
                          <span className="h-2 w-2 rounded-full bg-orange-500/75" />
                        ) : (
                          <span className="h-2 w-2 rounded-full border border-gray-300 bg-transparent
    | transition-colors group-hover:border-gray-400" />
                        )}
                      </span>

                      <div className="min-w-0 flex-1">
                        <p
                          className={`text-sm font-semibold transition-colors ${
                            isUnread ? "text-gray-900" : "text-gray-800"
                          }`}
                        >
                          {notif.title}
                        </p>

                        <p className="mt-0.5 text-sm break-words text-gray-500">
                          {notif.content}
                        </p>

                        {notif.application_id && (
                          <p className="mt-1 text-sm font-medium text-orange-600">
                            Open application
                          </p>
```

```
                          )}
                          <p className="mt-1 text-xs text-gray-400">
                            {notif.created_at &&
                              (isToday(new Date(notif.created_at))
                                ? `Today at ${formatTime(notif.created_at)}`
                                : formatDateTime(notif.created_at))}
                          </p>
                        </div>

                        {isUnread && (
                          <span className="mt-1 hidden shrink-0 text-xs font-semibold text-orange-500 group-
    | hover:block">
                            {isThisNotifLoading
                              ? "Marking as read..."
                              : "Mark read"}
                          </span>
                        )}
                      </div>
                    </Button>
                  </li>
                );
              })
            )}
          </ul>
        </div>
      </div>
    );
}
```

```
================================================================================================
FILE: src/services/application/get-user-applications.ts
================================================================================================
0001: import { supabaseClient as supabase } from "@/lib/supabase";
0002:
0003: export const getUserApplicationIds = async function() {
0004:     const { data: { user } } = await supabase.auth.getUser();
0005:     const userId = user?.id;
0006:
0007:     if (!userId) {
0008:         throw new Error("Cannot find user id");
0009:     }
0010:
0011:     const {data, error} = await supabase
0012:     .schema("private")
0013:     .from("inventors")
0014:     .select("application_id")
0015:     .eq("techgen_id", userId)
0016:
0017:     if (error) {
0018:         throw new Error(error.message);
0019:     }
0020:
0021:     return data;
0022: }
```

```
================================================================================================
FILE: src/services/views/get-dashboard-analytics-techgen.ts
================================================================================================
0001: import { supabaseClient as supabase } from "@/lib/supabase";
0002:
0003: export const getDashboardAnalyticsTechgen = async function (
0004: ) {
0005:   const { data: {user}} = await supabase.auth.getUser();
0006:
0007:   if (!user) {
0008:     throw new Error("User not found.");
0009:   }
0010:
0011:   const { data, error } = await supabase
0012:     .from("v_dashboard_analytics_techgen")
0013:     .select()
```

```
0014:     .eq("techgen_id", user.id)
0015:     .order("year", { ascending: false });
0016:
0017:   if (error) {
0018:     throw new Error(error.message);
0019:   }
0020:
0021:   return data;
0022: };
```

```
================================================================================================
FILE: src/hooks/views/useGetDashboardAnalyticsTechgen.ts
================================================================================================
0001: import { getDashboardAnalyticsTechgen } from "@/services/views/get-dashboard-analytics-techgen";
0002: import { useQuery } from "@tanstack/react-query";
0003:
0004: export function useGetDashboardAnalyticsTechgen(
0005: ) {
0006:   const { data, isPending } = useQuery({
0007:     queryKey: ["dashboard-analytics-techgen"],
0008:     queryFn: () => getDashboardAnalyticsTechgen(),
0009:   });
0010:
0011:   return { data, isLoading: isPending };
0012: }
```

```
================================================================================================
FILE: src/lib/roles.ts
================================================================================================
0001: export const ROLE_CONFIG = {
0002:   admin: {
0003:     home: "/admin",
0004:     allowedPrefixes: ["/admin"],
0005:   },
0006:   "up-official": {
0007:     home: "/up-official",
0008:     allowedPrefixes: ["/up-official"],
0009:   },
0010:   techgen: {
0011:     home: "/techgen",
0012:     allowedPrefixes: ["/techgen"],
0013:   },
0014: };
0015:
0016: export type Role = keyof typeof ROLE_CONFIG;
0017: export const VALID_ROLES = Object.keys(ROLE_CONFIG) as Role[];
```

```
================================================================================================
FILE: src/lib/constants/new-application.ts
================================================================================================
0001: export type NewApplicationFlowType = {
0002:   pageTitle: string;
0003:   pageSubtitle: string;
0004:   startPromptTitle: string;
0005:   startPromptDescription: string;
0006:   wizardCardTitle: string;
0007:   wizardCardDescription: string;
0008:   directCardTitle: string;
0009:   directCardDescription: string;
0010:   directMenuTitle: string;
0011:   directMenuDescription: string;
0012:   defaultDetailsTitle: string;
0013:   defaultDetailsDescription: string;
0014:   proceedLabel: string;
0015:   proceedHint: string;
0016:   toastTitle: string;
0017:   toastDescription: string;
0018: };
0019:
0020: export const TECHGEN_NEW_APPLICATION_COPY: NewApplicationFlowType = {
0021:   pageTitle: "New IP Application",
0022:   pageSubtitle:
```

```
    "Start a disclosure for your research output and prepare the required forms for submission.",
  startPromptTitle: "How would you like to start?",
  startPromptDescription:
    "You can let IRIS recommend a form or choose one directly if you already know the protection you need.",
  wizardCardTitle: "Yes, guide me through it",
  wizardCardDescription:
    "IRIS will ask a few short questions and suggest the most suitable disclosure form for your work.",
  directCardTitle: "No, I already know what to choose",
  directCardDescription:
    "Select the disclosure form directly if you already know the protection you want.",
  directMenuTitle: "Choose the disclosure form to use",
  directMenuDescription:
    "These are the standard forms used for different types of protection.",
  defaultDetailsTitle: "Form details",
  defaultDetailsDescription:
    "Select an option or complete the guide to see more details about the recommended disclosure form.",
  proceedLabel: "Proceed to application",
  proceedHint:
    "You can still review and complete the submission details in the next step.",
  toastTitle: "How this works",
  toastDescription:
    "IRIS will help you choose a starting disclosure form. You can either answer a few quick questions or
  | directly choose the form you need. You can still update the details later during review.",
};

export const ADMIN_NEW_APPLICATION_COPY: NewApplicationFlowType = {
  pageTitle: "Create IP Application",
  pageSubtitle:
    "Start an IP application record and prepare the required forms for endorsement or processing.",
  startPromptTitle: "How would you like to begin?",
  startPromptDescription:
    "You can let IRIS recommend a form or select one directly based on the intended protection.",
  wizardCardTitle: "Use guided classification",
  wizardCardDescription:
    "IRIS will ask a few short questions and suggest the most suitable disclosure form for the application.",
  directCardTitle: "Select a form directly",
  directCardDescription:
    "Choose the disclosure form directly if the protection type is already known.",
  directMenuTitle: "Choose the disclosure form to use",
  directMenuDescription:
    "These are the standard forms used for different types of IP protection.",
  defaultDetailsTitle: "Form details",
  defaultDetailsDescription:
    "Select an option or complete the guide to view details about the recommended disclosure form.",
  proceedLabel: "Proceed to application record",
  proceedHint:
    "Classification and application details can still be updated during review.",
  toastTitle: "How this works",
  toastDescription:
    "IRIS can help classify the application through a short guided flow, or you can select the form directly if
  | the protection type is already identified.",
};
```